Nu integration and the Gaia-on-ImSim plan

Status: plan; no source change yet. This document is the target model and the staged plan for two linked goals: integrate the vendored Nu game engine as a Scale host, and reimplement the Gaia editor on the public ImSim API. It is deliberately a plan, not a claim that either goal is implemented.

Goal

1. Make the Nu engine a first-class external host of the Scale monorepo without making it a Scale component.
2. Run Nu games from a Scale-owned base and publish an AOT game target.
3. Replace the vendored Gaia editor with a Scale-owned editor built on ImSim.
4. Keep the Nu engine and the original Gaia reference as an external, separately-licensed node.

Success criteria.

1. `just nu-build` builds the Nu core and a minimal ImSim game.
2. `just nu-smoke` starts an ImSim game through the host and exits cleanly. If a headless renderer is not available, the smoke is a bounded startup-and-exit test and UI runs are local/manual.
3. `bases/nu-game` runs a minimal ImSim game.
4. `bases/gaia-im-sim` loads a project fixture, shows the hierarchy, inspector, viewport, and console, selects a simulant, edits a property, saves, and plays in the editor.
5. The new editor declares its UI through ImSim dispatchers; direct world mutation is confined to a documented command adapter.
6. `projects/game` publishes an AOT game; `projects/gaia` is a desktop JIT editor.
7. A game screen can execute a Scale image unit through `components/nu-host` and the kernel C ABI.

What exists

Artifact	State	Relevance
`components/nu-engine`	Vendored Nu engine, 4,245 tracked files, external node. Its own `Nu.slnx`, `License.md`, and `Directory.Packages.props` opt-out.	The engine. It is the host, not a Scale component.
`components/nu-host`	10-line skeleton. No project reference to Nu.	The intended host seam. It must own the engine and Scale bridge.
`bases/nu-game`	README placeholder.	The intended ImSim game base.
`projects/game`	README placeholder.	The intended AOT game publish target.
`Nu.Gaia`	The original editor: `Gaia.fs` (4,622 lines), `GaiaPrelude.fs`, `GaiaPlugin.fs`, FSharp.Compiler.Service, ImGui, FSI code reload, project DLL loading, and direct world editing.	Behavioral reference and feature oracle. Do not copy it into Scale.
`WorldImSim` and the ImSim templates	`GameDispatcherImSim`, `ScreenDispatcherImSim`, `GroupDispatcherImSim`, `EntityDispatcherImSim`; static	The API the new editor must use. `Projects/Breakout ImSim` is the reference game.

Artifact	State	Relevance
	arguments with `|=` and `.=`, dynamic arguments with `@=`; `World.beginGame/endGame`, `beginScreen/endScreen`, `beginGroup/endGroup`, `doButton`, `doStaticModel`, subscriptions, and the entity/body/particle builders.	

The engine targets `net10.0` and imports `Nu.targets` into game projects. That targets file builds `Nu.Pipe`, processes assets, and copies SDL, Bullet, Assimp, and Vulkan dependencies. The host must treat the engine's build as a separate lane, not as part of the fast Scale gate.

Target layout

```
components/nu-host/        Nu engine bridge and Scale image bridge
bases/nu-game/             ImSim game base: game entry and configuration
bases/gaia-im-sim/         New Gaia editor base (JIT desktop)
projects/game/             AOT game publish target
projects/gaia/             Desktop editor publish target
```

`workspace.edn` gains the `gaia-im-sim` base and the `gaia` project. The external node stays `nu-engine`. `Scale.slnx` gains the new bases and projects as they land; it must never gain `Nu.slnx` or `Nu.Gaia.fsproj`.

Boundary decision required. The current Polylith rule permits only `components/nu-host` to reference the external node. ImSim application code inherits Nu dispatcher types and uses Nu lenses and builders directly, so a Scale base or editor component that cannot see Nu types cannot write an ImSim editor. The plan therefore needs one of two explicit decisions:

1. **Preferred: host-app exception.** Let `nu-host` remain the only component that references Nu, and let `bases/nu-game` and `bases/gaia-im-sim` reference the external node as host composition roots. Extend `tools/polylith-check.fsx` with exactly those two edges; no other base may reference Nu.
2. **Alternative: one host component.** Put the ImSim app code inside `components/nu-host` and re-export the needed Nu types. This preserves the letter of the current rule but makes `nu-host` a very large component and couples every editor change to it.

The plan assumes the first option. It must be recorded as a decision before the first base references Nu.

Build and license constraints

1. **License gate.** Nu and the original Gaia are under the Nu Game Engine Noncommercial License. Scale's BSD-2-Clause-Patent/CC0 licenses do not cover the vendored files. The engine stays external and keeps its notices. A Scale-owned editor is a clean-room implementation against the public ImSim API; it must not copy `Nu.Gaia` source. Whether the new editor's UX is a derivative work, and whether it may be distributed under Scale's license, is an owner/legal decision. Do not distribute the editor until that decision is recorded.
2. **F# Core alignment.** `Nu.fsproj` pins F# Core 10.1.204 and the game template's `Nu.targets` disables the implicit F# Core reference; Scale's kernel graph uses 10.1.401. A project reference from the host can unify to the higher version or warn. Prove the resolution with the first host build, pin one version for the whole host graph, and record any patch to the vendored engine. Prefer `PackageReference Update` in the host/base over editing the external node.

3. **Nu build lane.** Nu builds Nu.Pipe, assets, native SDL/Bullet/Assimp dependencies, and a Vulkan renderer. The lane is opt-in and must not be part of the fast Scale test gate. The CI lane may compile without a GPU; a windowed smoke is local or uses a dummy video driver only after that is proven.
4. **License and vendor drift.** Record the Nu commit or version in a `NU_VERSION` file or the plan's decision record. Do not silently update the engine: an engine update is a host-integration work item with its own smoke and license review.

ImSim mapping for the new Gaia

The editor is an ImSim game. State is long-lived; declarations are per frame.

Gaia concern	ImSim construction	Notes
Editor state	`GaiaEditorState` on the game; static arguments via `|=/.=`, dynamic values via `@=`	Project path and editor mode are reinitializing/static; selection, camera, dirty, play, and undo stack are dynamic. Never hold state only in a `Process` local.
Top-level flow	`GaiaGameDispatcher : GameDispatcherImSim`	Declares the start/editor/error screens each frame and drives project open/close.
Editor screens	`Start`, `Editor`, `NewProject`, `Error`	The `Editor` screen is the main overlay; the project's own screens are declared by the loaded project dispatcher.
Panels	`Toolbar`, `Hierarchy`, `Inspector`, `Viewport`, `Assets`, `Console`, `Dialogs` groups and entities	`World.doButton`, `doText`, `doPanel`, and custom ImGui draw callbacks for the viewport and reflection inspector.
Selection	`doSubscriptionToSelectionEvents`, entity selection, and an editor selection value	Selection is a value; the editor does not mutate the project's selected simulant directly outside a command.
Commands	an `EditorCommands` module behind the UI	Create, delete, duplicate, transform, property set, undo, redo, save, and play are commands returning a state update or a named refusal.
Project load	the Nu project/plugin loading path	Validate the assembly/plugin, load its packages/assets, declare its dispatchers, and surface load errors as editor dialogs.
Play in editor	`World.setTimeAdvancing`, screen visibility, and a play/pause state	The editor overlay hides or becomes non-interactive; selection maps to the running game's simulants; stop restores the edited snapshot.
Code reload	ImSim reinitializing arguments plus the Nu/FSI reload path	Editor state survives; the edited game's reload is a separate, JIT-only operation. The editor itself is not an AOT target.
Persistence	a versioned editor-state file	Store project path, layout, selection, and play mode; do not depend on Gaia's private state file format.

The viewport and the property inspector are the two places where direct engine or ImGui work is expected. They must be isolated behind a small view: `EditorViewport.draw world` and `EditorInspector.draw selection world`. The rest of the editor should read as an ImSim declaration.

Phases

Phase 0: Baseline and build lane

1. Add `just nu-build` to build `components/nu-engine/Nu/Nu/Nu.fsproj` and the ImSim template.
2. Add a `just nu-smoke` that starts the template game, runs a bounded number of frames, and exits; determine whether Nu supports a headless/dummy SDL mode.
3. Prove F# Core resolution and the Nu.Pipe/asset build.
4. Record the Nu version and license notice.
5. Acceptance: the Nu core and template build; the smoke exits zero; no Scale component references Nu except the host.

Phase 1: Host game shell

1. Replace the `nu-host` skeleton with a small host API: `NuHost.start`, `NuHost.runGame`, `NuHost.runEditor`, and `NuHost.loadProject`.
2. Add `bases/nu-game` with the minimal ImSim template game, referencing the host and importing `Nu.targets`.
3. Add a `--smoke` mode that initializes the world and exits after a bounded frame count.
4. Acceptance: the minimal game starts, processes a frame, and exits; the `poly-check` boundary is explicit; the license notice is present.

Phase 2: Editor shell and read-only project view

1. Add `bases/gaia-im-sim` with `GaiaGameDispatcher`, `GaiaEditorState`, and the `Start`/`Editor` screens.
2. Implement project selection, load, hierarchy, selection, inspector, viewport, and console as read-only views.
3. Use an editor fixture: a small Nu project with a screen, group, and entity.
4. Acceptance: the editor opens the fixture, lists the hierarchy, selects a simulant, and renders its properties; a missing or invalid project is a named dialog, not a crash; UI declarations are ImSim, not imperative panel code.

Phase 3: Editing, persistence, and undo

1. Add `EditorCommands`: create, delete, duplicate, rename, property set, transform, save, and undo/redo.
2. Persist the editor state and the project; keep dirty state and a save prompt.
3. Move the hierarchy and inspector from read-only to editable.
4. Acceptance: a property edit round-trips through save/load; create/delete and undo/redo restore the expected selection and values; a protected/imSim-created simulant is a named refusal, not a destructive edit.

Phase 4: Play in editor and code reload

1. Add play/pause/stop, time advancing, selection mapping, and the editor-overlay behavior.
2. Add the Nu code-reload path; use ImSim reinitializing arguments for editor state that must survive.
3. Keep the old Gaia available as a behavioral oracle only.
4. Acceptance: edit, play, observe, stop, and resume editing; a code reload preserves the project/selection/layout; failures are dialogs with named reasons.

Phase 5: Panel completion and retirement of the old editor path

1. Move the asset browser, project creation, dialogs, log, and remaining controls into ImSim.
2. Add snapshot/parity checks for the major panels.
3. Route the new editor through the host and stop referencing the vendored Gaia assembly from the new editor.
4. Acceptance: a feature-parity checklist for the agreed editor scope; no `Nu.Gaia` reference in the new editor; the old Gaia remains untouched in the external node.

Phase 6: Scale image bridge and AOT game

1. Implement the host callbacks and load the Scale kernel through the C adapter.
2. Expose a minimal Scale call from an ImSim screen; swap a Prelude image in the game loop.
3. Publish the game target with NativeAOT and run the C conformance harness.
4. Acceptance: the game screen executes a Scale image unit and uses its result; image swap works; the AOT game smoke passes; the kernel AOT conformance is green.

Test strategy

1. **Engine lane.** `nu-build` and `nu-smoke`: compile the Nu core and the template; start and exit. This is not part of the default Scale gate.
2. **Host lane.** A minimal game through `nu-host`: startup, one frame, shutdown; a missing/invalid project is a named refusal.
3. **Editor lane.** Pure tests for `GaiaEditorState` and `EditorCommands` (selection, dirty state, undo/redo, property set, save/load round-trip); an integration fixture that loads a small project and asserts hierarchy and inspector output; snapshot the panel text where stable.
4. **Play/reload lane.** Edit, play, stop, reload, and re-select; the editor state survives; a failed reload is reported.
5. **Scale bridge.** Kernel AOT + C conformance; an image unit called from a game screen; image swap; AOT game smoke.
6. **Licensing.** A build-time notice check and a review gate; the editor is not published until the license decision is recorded.

A GPU-free test path is a design requirement for CI. If Nu cannot run without a window/GPU, the CI gate is compile + a bounded process smoke, and the UI tests are local. Do not pretend a skipped UI test is coverage.

Risks and mitigations

Risk	Mitigation
Nu/Gaia noncommercial license	Keep Nu external; clean-room editor; owner/legal decision before release; preserve notices; no copying of `Nu.Gaia` source.
Nu build is heavy and native	Separate `nu-*` lane; no default-gate impact; pin the engine version; document SDL/Vulkan/Assimp/Bullet prerequisites.
No headless renderer	Compile + bounded startup smoke in CI; local UI runs; prove a dummy SDL mode before claiming headless UI coverage.
F# Core version conflict	Prove the host graph first; pin one version; prefer a host override over patching the vendor; record any patch.

Risk	Mitigation
ImSim state is re-declared every frame	Long-lived state in game properties/static/dynamic args; pure state transitions; never in `Process` locals.
Editor and game share one world	Use a dedicated editor screen/group and a play state; validate screen and input ownership early; do not create a second renderer without a design.
Code reload and FSI are JIT-only	Keep the editor JIT; keep the AOT game separate; a reload failure is a dialog, never a crash.
Gaia parity is open-ended	Agree the first-release editor scope before Phase 2; treat the old Gaia as a read-only oracle, not a source.
Protected simulants and engine invariants	Commands return named refusals; never bypass protection; test the refusal paths.

Decisions to record before implementation

1. The Polylith host-app exception: may the two Nu host bases reference the external node directly, or must all Nu use stay inside `nu-host`?
2. The license position for the new editor and the game base.
3. The first-release editor scope: viewer, viewer plus editing, or full Gaia parity.
4. The pinned Nu version and update policy.
5. One world with an editor overlay versus a separate editor/game world.
6. The editor state file format and whether Gaia's old state file is imported.

Non-goals

1. Do not modify the vendored Nu engine or `Nu.Gaia` as part of the first editor slices.
2. Do not port the editor to NativeAOT; the editor is a desktop JIT tool.
3. Do not replace the Nu renderer, physics, or asset pipeline.
4. Do not claim full Gaia parity until the agreed scope is met.
5. Do not make the Nu engine or the editor a Scale component.

Definition of done

The plan is done when the host/game base, the new ImSim editor with the agreed feature scope, the AOT game target, and the Scale image bridge all have executable checks and documented boundaries, and the license decision is recorded. Until then, the Nu node remains external and the old Gaia remains the reference.